

standard in most binaries distributed by MySQL AB, the exception being some OEM versions.

The advantages

1. InnoDB should be used where data integrity is a priority because it inherently takes care of this aspect with the help of relationship constraints and transactions.
2. It is faster at write-intensive (inserts, updates) tables because it uses row-level locking, and only holds up changes to the same row that data is being inserted in or updated.

The disadvantages

1. Because InnoDB has to take care of the different relationships between tables, database administrators and scheme creators need to spend more time in designing the data models, which are more complex than those of MyISAM.
2. Consumes more system resources like RAM. It is recommended that if there is no substantial need for it after the installation of MySQL, the InnoDB engine be turned off.
3. No full-text indexing.

The differences between MyISAM and InnoDB

The main difference between the two is support for 'referential integrity' and 'transactions'. Other differences are based on locking, roll-backs, and full-text searches.

Referential integrity: This ensures that relationships between tables remain consistent. For example, if a table consists of a foreign key that is pointing to another table, then any updates, deletions or changes occurring in the pointed table must cascade to the linking table. InnoDB is a relational DBMS (RDBMS) and thus has referential integrity, while MyISAM does not.

Transactions and atomicity: Data in a table gets manipulated

by using DML (Data Manipulation Language) operations like UPDATE, INSERT, DELETE. This means that any modification done by two or more DML statements should be handled as a single unit of work, so that either all the changes are applied or none are. MyISAM does not support transactions whereas InnoDB does. If any operations are interrupted while in progress, then operations get aborted and the rows that get affected will remain affected. InnoDB has the atomicity feature, so rows that get affected will be rolled back.

Locking table: If a query runs on a MyISAM table, then the entire table gets locked. Any subsequent query will not run if the previous query is not finished. In InnoDB, on the other hand, only the rows being used by the current running query are locked. The rest of the table can be used for subsequent queries.

Transactions and rollbacks: When any query runs on a MyISAM table, the changes are committed. But in InnoDB, those changes can be rolled back by using commands like ROLLBACK. Changes can be saved by using the command SAVEPOINT, and changes are committed with a command like COMMIT.

When to use one or the other

InnoDB works well on any site with a DB that is frequently being modified (rather than MyISAM) by resolving table-locking bottlenecks. However, MyISAM performs better than InnoDB when using large tables that require vastly more select operations than any other DML operations. This is because locking the entire table is quicker than figuring out which rows are to be locked in the table. The larger

the table, the more time it will take for InnoDB to figure out which rows are not accessible. On the other hand, InnoDB performs better when data within tables changes frequently. If a table changes write data more than read data per second, then InnoDB can keep up with large amounts of requests more easily than locking the entire table for each one. If your database uses foreign key constraints or if it needs support for transactions (i.e., when modifications are being done by two or more DML operations handled as a single unit of work, either all the changes are applied or all of them are reverted) then you can choose the InnoDB engine. Another difference is concurrency. When using MyISAM, a DML statement will obtain an exclusive lock on the table and while that lock is held, no other session can perform a SELECT or any other DML operation on the table. Second, if you are working with small volumes of data that is static, then MyISAM can have a performance advantage over InnoDB. If your data is getting updated, but infrequently, MyISAM has an advantage over InnoDB. But when to select one over the other is a subjective decision as neither of them is perfect in all situations. There are advantages and disadvantages to both storage engines.

So, we can conclude that InnoDB is more suitable for data critical situations that require frequent inserts or any DML operation on the DB. MyISAM, on the other hand, performs better with applications that have static data. It doesn't depend on data integrity and mostly uses the SELECT operation to display the data. **END** 🐧

References

<https://wikipedia.org>

By: Neetesh Mehrotra

The author works at NIIT Technologies as a senior test engineer. His areas of interest are Java development and automation testing.